

Script Reference & User Guide

Comparative Evaluation of Vision-Based Finger Tracking for Piano Interaction

1. Pipeline Overview

The system is structured as a sequential pipeline. Each script hands off its output to the next stage. `live_preview.py` is a standalone tool that does not persist any data.

<code>key_calibration.py</code>	run once per camera/tripod position
<code>record.py</code>	capture video + MIDI per participant
<code>recalibrate.py</code>	(optional) fix calibration on saved video
<code>analyse.py</code>	compute MPJPE results per session
<code>plot_results.py</code>	generate group charts and participant reports
<code>live_preview.py</code>	standalone live test (no data written to disk)
<code>data_summary.py</code>	audit data/raw/ session counts at any time

2. config.py

Purpose: Central store for every tunable constant in the project. All other scripts import this instead of hard-coding values.

Depends on: nothing (no internal imports)

Used by: all scripts

Key constants

Constant	Description
CAP_WIDTH / HEIGHT / FPS	Camera capture resolution and target frame rate
RESIZE_WIDTH / HEIGHT	Resolution MediaPipe runs inference at (smaller = faster)
MODEL_COMPLEXITY	MediaPipe model complexity (0 = lite, 1 = full)
MIN_DETECTION_CONFIDENCE	Minimum score to accept a hand detection
MIN_TRACKING_CONFIDENCE	Minimum score to continue tracking a detected hand
LUX_DIM_THRESHOLD	Below this value (lux) the session is labelled Dim
LUX_BRIGHT_THRESHOLD	At or above this value the session is labelled Bright
HAND_SIZE_MIN / MAX_CM	Validation range for the manually entered hand length
HAND_SIZE_SMALL_MAX_CM	Hand lengths below this are categorised as Small
HAND_SIZE_LARGE_MIN_CM	Hand lengths at or above this are categorised as Large
ACCURACY_THRESHOLD_RATIO	Fraction of white key width used as the accuracy pass threshold
MASK_*	Fill and border colours for the keyboard overlay drawn on-screen

3. Support Scripts

These scripts are imported by the main pipeline scripts. They are not run directly.

3.1 camera_setup.py

Purpose: **Opens** the camera with the correct backend for the current OS and applies the capture resolution and FPS from config. Returns a ready VideoCapture object.

Depends on: config, cv2

Used by: record.py, key_calibration.py, live_preview.py

Key function:

- `init_camera(config)` — opens the camera using CAP_DSHOW on Windows (required to avoid DirectShow double-open conflicts), sets width/height/FPS and buffer size, returns the VideoCapture handle.

3.2 lux_calculator.py

Purpose: Converts a numeric lux value into a human-readable lighting category label.

Depends on: config

Used by: record.py, live_preview.py

Key function:

- `lux_to_label(lux)` -> "Dim" / "Indoor" / "Bright" (thresholds are set in config.py)

3.3 fitzpatrick_detector.py

Purpose: Estimates the participant's Fitzpatrick skin type (I-VI) from a live video frame using the ITA (Individual Typology Angle) colorimetric method. Samples CIELab pixels from the palm convex hull (landmarks 0,1,5,9,13,17). Runs once per session at setup time.

Depends on: config, cv2, numpy

Used by: record.py

Key functions:

- `detect_skin_type(frame, hand_landmarks, fw, fh)` -> (fitzpatrick_type, label, ita_angle) or (None, None, None) if palm not visible
- `ita_to_fitzpatrick(ita)` -> (type_int, label_str) — maps ITA angle to the six Fitzpatrick categories using Chardon (1991) thresholds

3.4 stats_collector.py

Purpose: Rolling statistics tracker for the recording/preview loop. Computes FPS, MediaPipe inference time, and total frame latency over a configurable window.

Depends on: numpy, collections.deque

Used by: record.py, live_preview.py

Key class: StatsCollector(warmup_frames, stat_frames)

Method / property	Description
update(loop_start, inference_ms, latency_ms)	Call once per frame to push new measurements
fps	Rolling average frames per second
inference_ms	Rolling average MediaPipe inference time (ms)
latency_ms	Rolling average total frame latency (ms)
warmup_done	True once warmup_frames have elapsed
print_final_stats(w, h, complexity)	Prints a summary table on session exit

3.5 midi_recorder.py

Purpose: Threaded MIDI input capture with time.perf_counter() timestamps. All timestamps are relative to a shared t0 set at recording start (SPACE keypress), keeping video and MIDI in sync.

Depends on: mido, python-rtmidi, json, threading, pathlib

Used by: record.py, live_preview.py

Key classes:

Class / function	Description
MidiRecorder.start(t0)	Begins background daemon thread polling the MIDI port every 1 ms. Skips realtime messages (clock, active sensing).
MidiRecorder.stop()	Joins the background thread.
MidiRecorder.save(out_dir, pid_str)	Writes p###_midi.jsonl (one JSON line per event) and p###_midi.mid (standard MIDI file, type 0, 120 BPM).
FrameLogger.log()	Call once per writer.write(frame). Records (frame_index, time_s) relative to t0.
FrameLogger.save(out_dir, pid_str)	Writes p###_frames.csv.
list_midi_input_ports()	Returns list of detected MIDI port name strings.

3.6 model_manager.py

Purpose: Abstract interface and concrete wrappers for switching between hand-tracking models at runtime. Defines a common API (load, infer, draw, close) so analyse.py can run either model without branching logic.

Depends on: config, cv2, numpy, mediapipe

Used by: live_preview.py, analyse.py

Concrete model classes:

- `MediaPipeHandModel` — wraps `mp.solutions.hands`. `infer()` returns the standard MediaPipe result object.
- `OpenPoseHandModel` — loads pre-trained Caffe weights via `cv2.dnn`. `infer()` extracts 21 keypoints from heatmap outputs. Operates on a keyboard-area crop rather than the full frame.

4. `key_calibration.py`

Purpose: Interactive GUI for aligning a virtual piano keyboard overlay over the camera view. The overlay geometry (position, key width/height, perspective warp) is saved to `data/calibration/key_centers.json` and loaded by all downstream scripts.

Depends on: `config`, `camera_setup`, `cv2`, `numpy`, `json`, `math`

Used by: `record.py`, `live_preview.py`, `analyse.py`, `recalibrate.py`

Key class: `KeyMask`

Method	Description
<code>draw(frame, center_viz='line')</code>	Blends the perspective-warped key overlay onto a frame in-place. <code>center_viz='line'</code> draws a full-depth vertical line per key; <code>'dot'</code> draws a single point at the calculated centre.
<code>on_mouse(event, x, y, flags)</code>	Handles drag-to-move, edge-drag-to-resize, corner-drag-to-warp, scroll-to-add/remove keys.
<code>save(cal_dir, fw, fh)</code>	Writes <code>key_centers.json</code> with layout parameters and pre-computed per-key centre coordinates.
<code>KeyMask.load(path)</code>	Reconstructs a <code>KeyMask</code> from a saved JSON file.
<code>KeyMask.default(fw, fh)</code>	Returns a default 29-key layout sized to ~3/4 frame width.

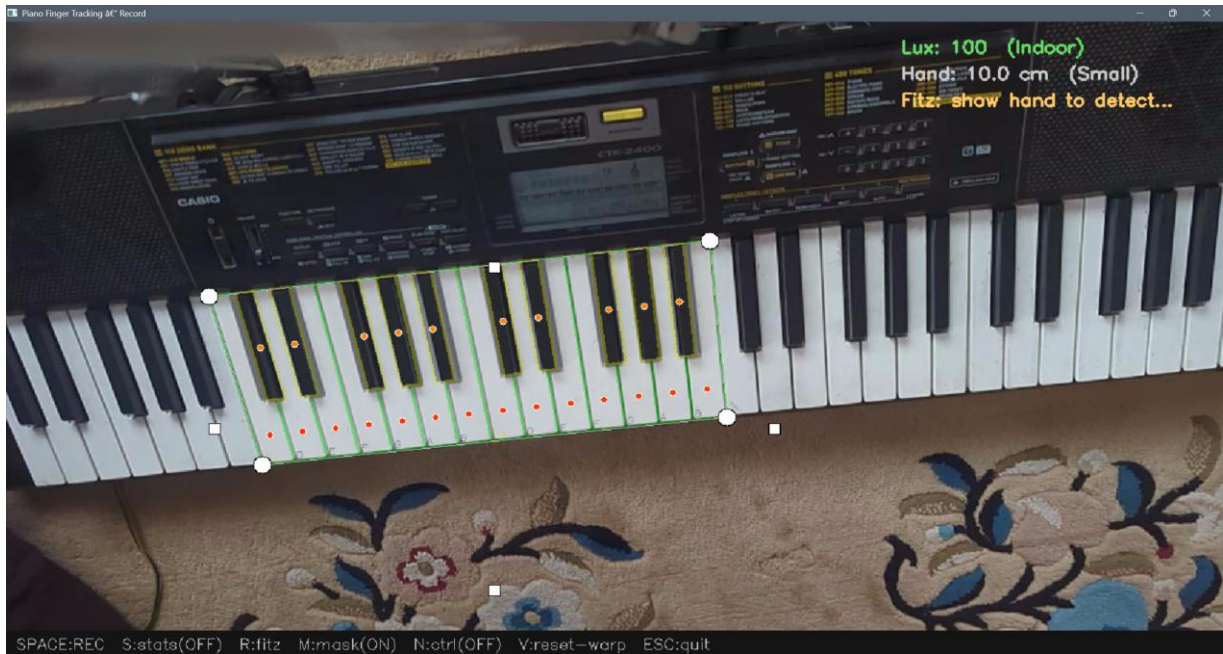
Exported utility function:

- `build_key_layout(start_midi, num_white, ox, oy, wkw, wkh)` — re-derives per-key centre coordinates from saved parameters; used by `analyse.py` and `recalibrate.py`.

Controls

Key / Action	Effect
Drag interior	Move the entire keyboard overlay
Drag left / right edge	Resize key width (opposite edge stays fixed)
Drag bottom edge	Resize key height
Drag corner circle	Warp perspective to correct camera angle
Scroll wheel	Add or remove one key on the right end
[/]	Shift the starting MIDI note down / up

-/=	Fine key width -1 / +1 px
C	Toggle centre visualisation (line / dot)
F	Toggle 180 degree flip to match record.py orientation
H	Toggle on-screen help
V	Reset perspective warp to rectangle
ENTER or S	Save calibration to key_centers.json
ESC	Quit without saving



key_calibration.py — keyboard overlay aligned to physical piano keys on camera feed

INFO Run `key_calibration.py` first, or any time the camera or tripod position changes. All downstream scripts rely on the saved `key_centers.json`.

5. record.py

Purpose: Main data collection script. Displays a live MediaPipe hand-tracking preview with the key overlay active. Before each participant, a setup screen collects participant ID, Fitzpatrick skin type, ambient lux, and hand size. Pressing SPACE starts and stops recording.

Depends on: `config`, `camera_setup`, `fitzpatrick_detector`, `lux_calculator`, `stats_collector`, `midi_recorder`, `key_calibration`, `mediapipe`, `cv2`, `numpy`, `mido`

Used by: (main script — not imported by others)

Output files per session (data/raw/p###/)

File	Contents
p###.mp4	Raw video with no overlays. 180 degree flip baked in if flip_y = True.
p###_frames.csv	frame_index, time_s — one row per recorded frame, timestamps relative to recording start.
p###_midi.jsonl	One JSON line per MIDI event with time_s offset from recording start.
p###_midi.mid	Standard MIDI file (type 0, 120 BPM, 480 ticks/beat).
p###_session.json	Session metadata: lux, hand size, Fitzpatrick type, flip state, MIDI port name, and references to all output file names.
p###_key_centers.json	Snapshot of the active calibration at the moment recording started.

Key controls

Key	Action
SPACE	Start recording / stop recording
N	Finish session and go to setup screen for next participant
M	Toggle key overlay visibility
P	Toggle mask control panel (move/resize overlay with buttons)
C	Toggle centre visualisation (line / dot)
S	Toggle FPS / inference / latency stats overlay
R	Re-run Fitzpatrick skin-type auto-detection
F	Toggle 180 degree frame flip
V	Reset mask perspective warp to rectangle
ESC	Quit

SESSION SETUP

Fill in both fields, then press ENTER

Lux Level

lux

Fitzpatrick Type (optional, 1–6 or blank = auto)

(1–6)

→ auto-detect (20-frame average)

Hand Size

cm

MIDI Input Port

No MIDI device found

(connect piano via USB and restart)

Last note: ----

Camera

Camera 0 (1280 x 720)

(only camera detected)

f: flip [off]

TAB: switch field

ENTER: confirm

[/]: MIDI port

c/C: camera

f: flip

ESC: quit

record.py setup screen — participant ID, Fitzpatrick type, lux, and hand-size input fields



record.py recording view — hand skeleton and key overlay active, REC indicator visible

6. live_preview.py

Purpose: Live hand-tracking preview and real-time MPJPE accuracy test. Mirrors record.py's display but writes nothing to disk. Shows live MPJPE, accuracy percentage, and detection rate in the top-right corner. On exit (ESC), displays a full results panel with per-finger breakdown.

Depends on: config, camera_setup, key_calibration, midi_recorder, stats_collector, mediapipe, PIL, cv2, numpy, mido

Used by: (standalone tool — not imported by others)

MPJPE computation

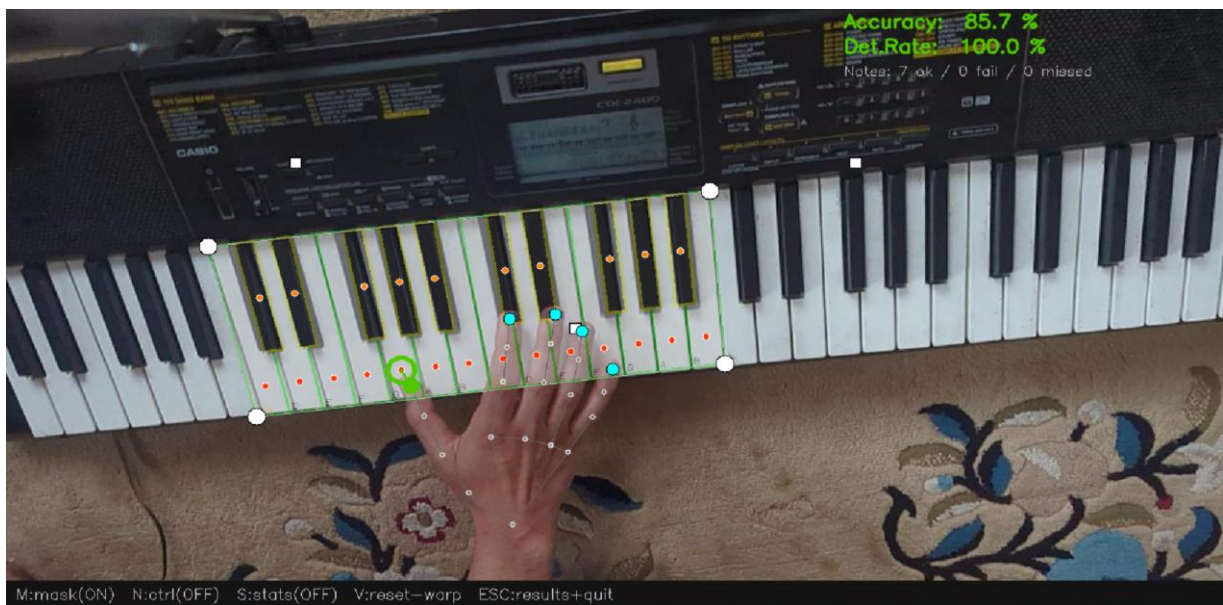
For every MIDI note_on event (velocity > 0) while at least one hand is visible, the script classifies the outcome into one of three categories:

Outcome	Meaning	Effect on MPJPE
matched	A fingertip landmark was inside the key polygon	Horizontal error $ tip_x - key_cx $ added to MPJPE
detection_fail	Hands visible but no fingertip tip inside the polygon	Counted separately — NOT added to MPJPE
missed	No hand landmarks detected at all	Counted separately — NOT added to MPJPE

INFO MPJPE uses horizontal-only error ($|tip_x - key_{cx}|$) to remove depth / finger-length bias from the measurement. Any fingertip Y position within the key height is considered correct.

Key controls

Key	Action
M	Toggle key mask overlay
N	Toggle mask control panel
C	Toggle centre visualisation (line / dot)
S	Toggle FPS / inference / latency stats overlay
V	Reset perspective warp to rectangle
ESC	Stop test and show results panel



live_preview.py — hand skeleton, key overlay, and live MPJPE readout top-right corner

MPJPE TEST RESULTS	
MPJPE (horizontal)	5.7 px (med 5.0)
Accuracy (<10 px)	85.1 %
Detection rate	95.7 %
Notes matched	134
Detection fails (no tip on key)	6
Notes missed (no hands)	0
Max error	15.0 px
Per-finger breakdown (L = lower keyboard half, R = upper keyboard half):	
Left Hand	Right Hand
Thumb 6.3px 76.9% (13)	Thumb 6.5px 75.0% (8)
Index 3.6px 100.0% (12)	Index 5.0px 95.0% (20)
Middle 2.5px 100.0% (12)	Middle 6.0px 95.2% (21)
Ring 3.4px 92.3% (13)	Ring 6.6px 85.7% (14)
Pinky 8.9px 60.0% (10)	Pinky 9.7px 45.5% (11)
Press any key to close	

live_preview.py results panel — MPJPE, accuracy %, detection rate, and per-finger table

7. recalibrate.py

Purpose: GUI tool for aligning the key overlay against a pre-recorded session video. Useful when the session calibration file was lost or was inaccurate. Saves a per-session p###_key_centers.json that analyse.py will prefer over the global calibration.

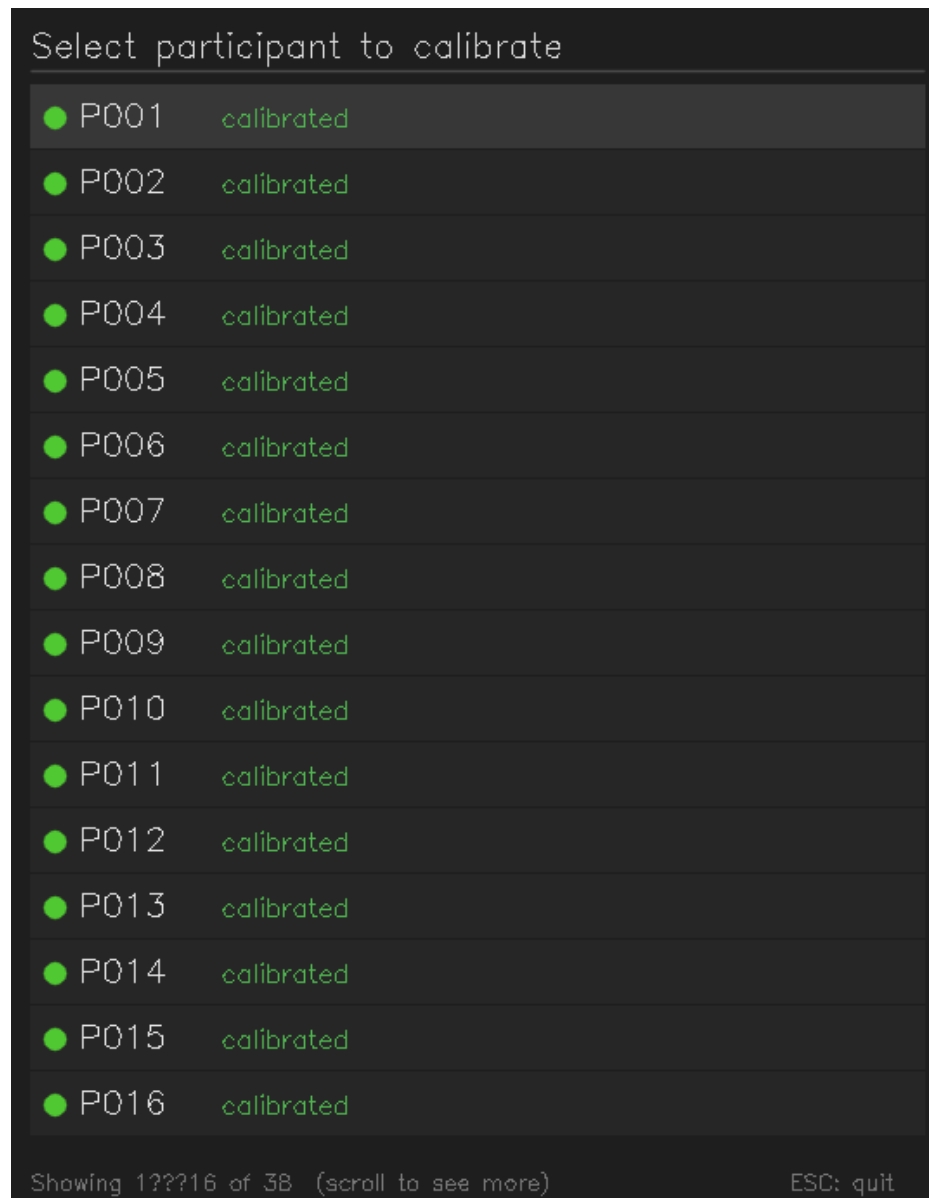
Depends on: key_calibration.KeyMask, cv2, numpy, json, csv

Used by: (standalone tool — run manually before re-analysing a session)

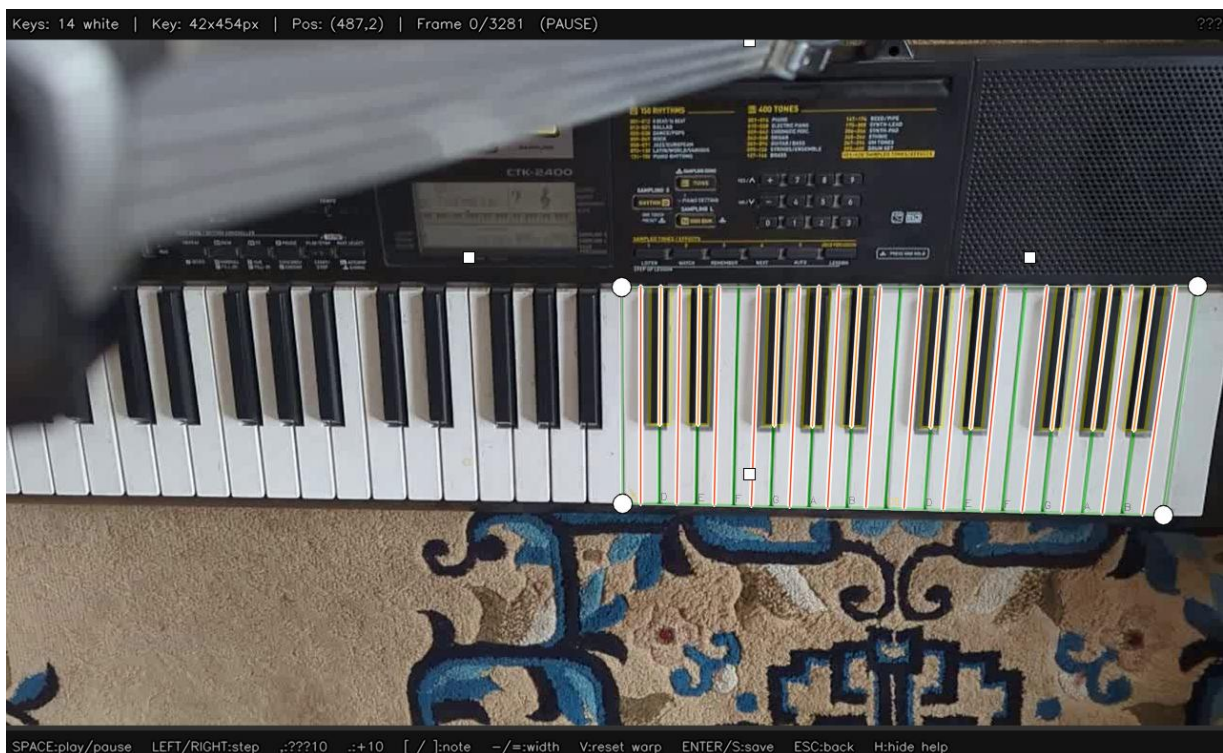
Workflow

1. Run `python -m scripts.recalibrate` to open the participant picker.

2. A scrollable list of all data/raw/p###/ folders is shown. Green dot = already has a per-session calibration. Blue dot = will use the global calibration.
3. Click a participant to open the calibration editor, which plays back that session's video.
4. Align the overlay to match the piano in the video. The NOW: C5 indicator shows which MIDI note was played at the current frame.
5. Press ENTER or S to save p###_key_centers.json and return to the picker.



recalibrate.py participant picker — scrollable list with green/blue status dots



recalibrate.py calibration editor — video playback with overlay and MIDI note indicator

8. analyse.py

Purpose: Offline per-session MPJPE computation. For each note_on event in the MIDI log, it seeks to the nearest video frame, runs the chosen model, and records the outcome. Supports MediaPipe and OpenPose via model_manager. Can be run from the command line or via a Tkinter GUI session picker.

Depends on: config, key_calibration.KeyMask, model_manager, mediapipe, cv2, numpy

Used by: (main analysis script — outputs consumed by plot_results.py)

Calibration priority

analyse.py looks for a per-session calibration file first (data/raw/p###/p###_key_centers.json), then falls back to the global calibration (data/calibration/key_centers.json). Use recalibrate.py to create per-session files where needed.

Hand classification

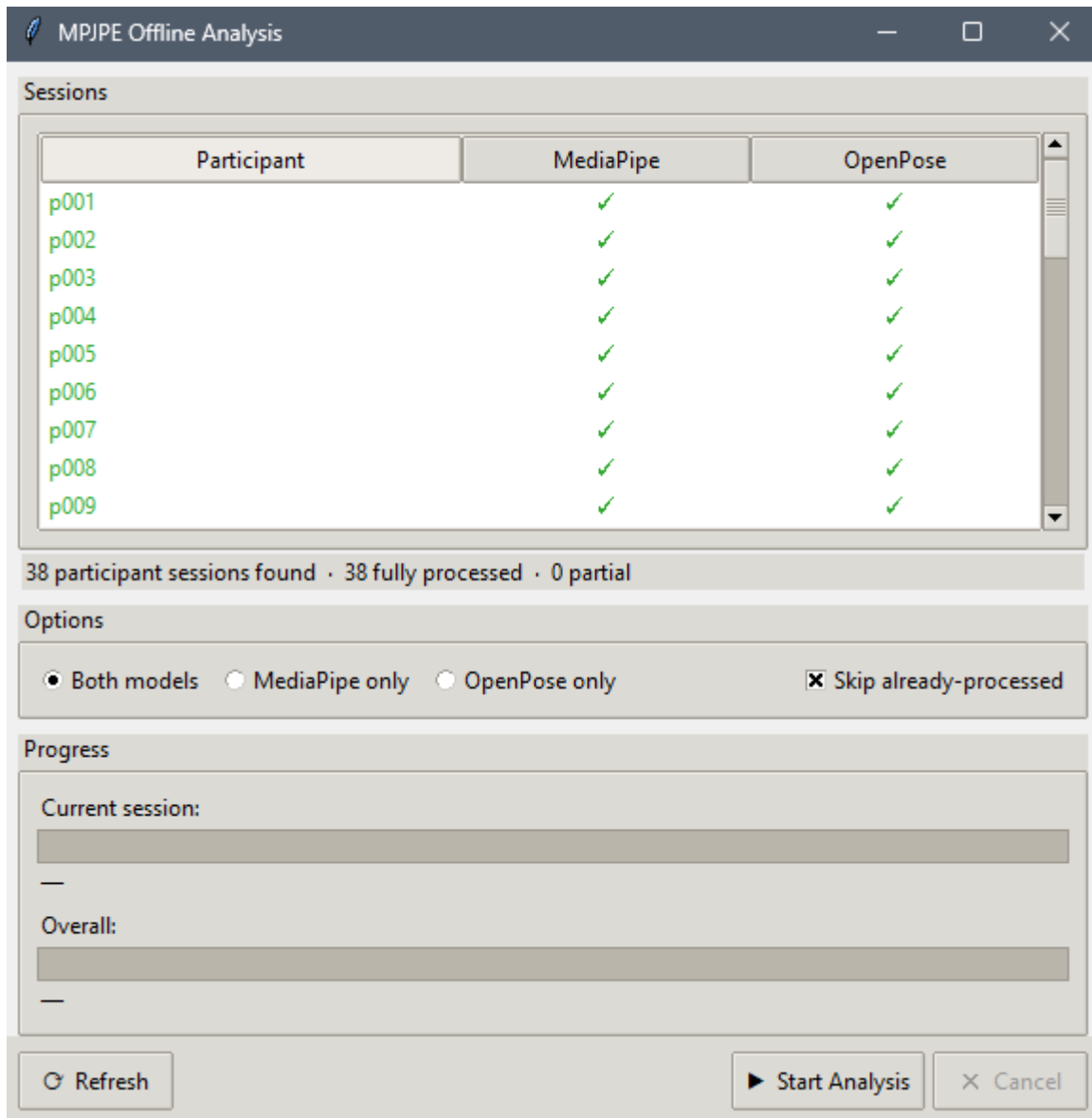
- MediaPipe: uses multi_handedness label combined with thumb-MCP-to-wrist geometry to determine physical L / R, accounting for mirrored camera setups.
- OpenPose: geometry is unreliable on scene-level crops; falls back to keyboard-position split (left of keyboard midpoint = L).

Output files (data/processed/p###/)

File	Contents
p###_mediapipe_results.json	Per-session MPJPE results: detection_rate_pct, per_hand L/R with matched, detection_fail, mpjpe_px, accuracy_pct, and per-finger arrays.
p###_openpose_results.json	Same structure for OpenPose (hand split uses keyboard-position fallback).

Usage

```
python -m scripts.analyse                # open Tkinter GUI
python -m scripts.analyse data/raw/p001/  # process one session (both
models)
python -m scripts.analyse data/raw/p001/ --model mediapipe
python -m scripts.analyse data/raw/p001/ --model openpose
```



analyse.py Tkinter GUI — session list with MediaPipe and OpenPose status columns

9. plot_results.py

Purpose: Reads all processed result JSON files under data/processed/ and generates publication-ready group-level charts plus individual participant reports in data/plots/.

Depends on: matplotlib, numpy, json, pathlib

Used by: (terminal script — outputs are PNG files for the report)

Generated charts (data/plots/)

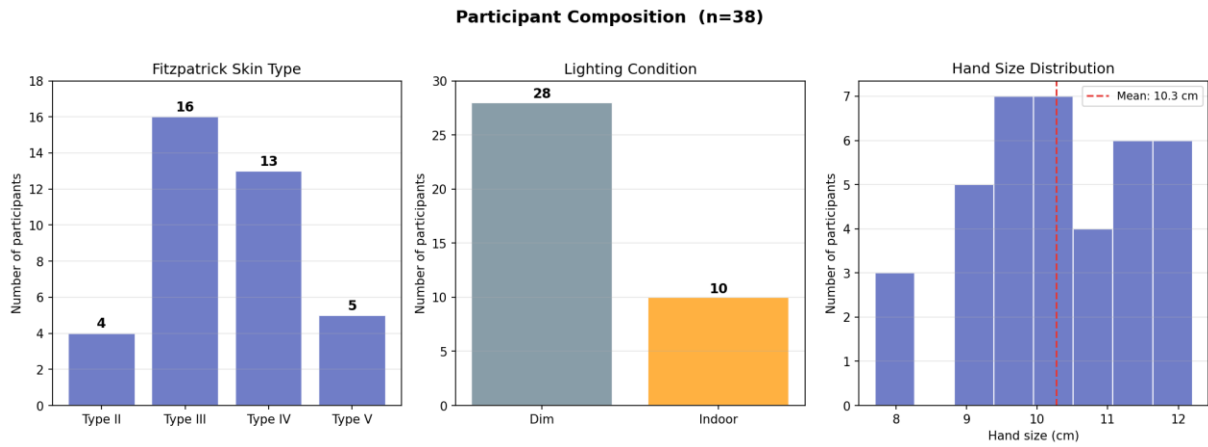
File	Description
01_model_comparison_mpjpe.png	Bar chart: MediaPipe vs OpenPose MPJPE per participant
02_per_finger_mpjpe.png	Mean per-finger MPJPE: MediaPipe L, MediaPipe R, OpenPose combined
02_per_finger_mpjpe_mediapipe.png	MediaPipe-only per-finger MPJPE: L and R panels
03_detection_breakdown.png	Stacked bar: matched / detection-fail / missed proportions per model
04_mpjpe_by_lux.png	MPJPE grouped by lighting condition (Dim / Indoor / Bright) with SD error bars
05_mpjpe_by_fitzpatrick.png	MPJPE grouped by Fitzpatrick skin type with SD error bars
06_mpjpe_vs_handsize.png	Scatter: MPJPE vs hand size (cm) with regression lines per model
07_finger_distribution_mediapipe.png	Box plots of per-finger MPJPE distribution across all sessions (MediaPipe)
07_finger_distribution_openpose.png	Same for OpenPose (combined L+R)
08_heatmap_mediapipe.png	Heatmap: participants x fingers, 10 columns (L/R split)
08_heatmap_openpose.png	Heatmap: participants x fingers, 5 columns (combined)
reports/p###_report.png	Individual participant report (see below)

Participant report layout

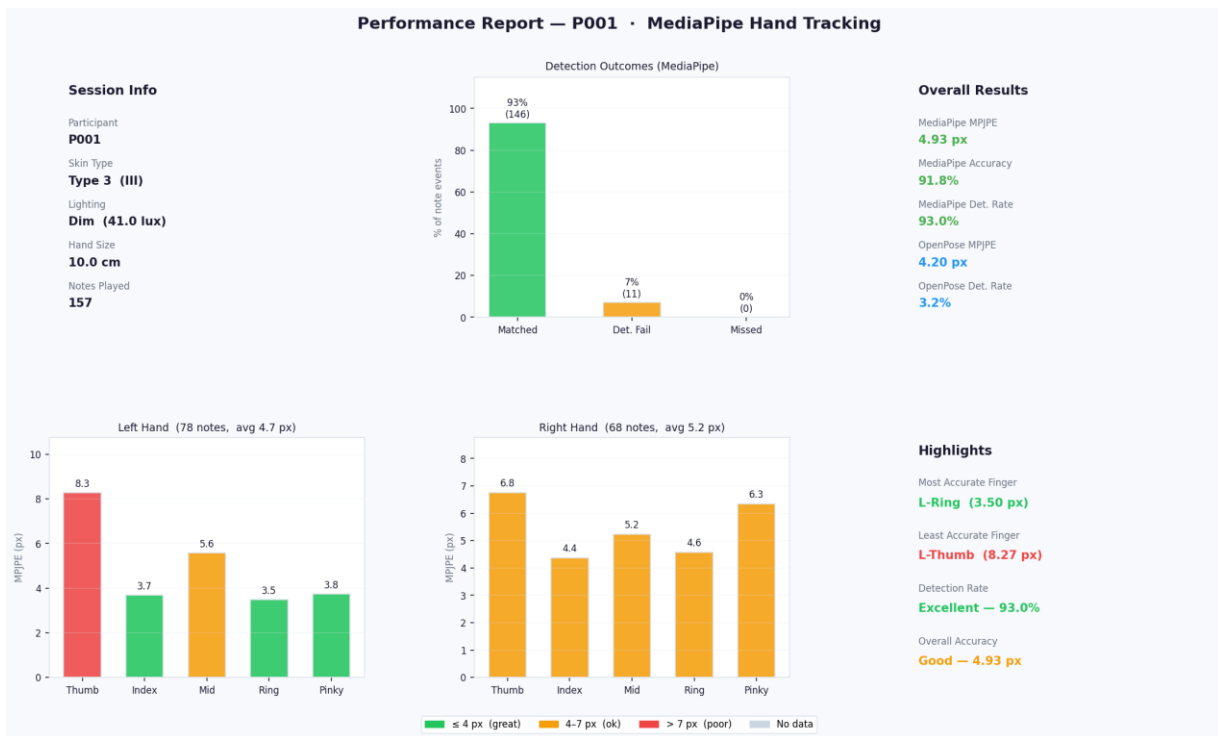
Panel position	Contents
Top left	Session info: PID, Fitzpatrick type, lighting, hand size, notes played
Top centre	Detection outcome bar chart (Matched / Det.Fail / Missed)
Top right	Overall results: MPJPE, accuracy %, detection rate for both models
Bottom left	Per-hand bar chart (MediaPipe) coloured by accuracy tier
Bottom centre	Per-hand bar chart (OpenPose) coloured by accuracy tier
Bottom right	Highlights: best/worst finger, detection rate rating, overall accuracy rating

Usage

```
python -m scripts.plot_results          # all sessions
python -m scripts.plot_results --pid p001 p003  # specific sessions
python -m scripts.plot_results --out figures/    # custom output folder
```



Example generated group chart — diversity chart



Example participant report — p001_report.png with all 6 panels

10. data_summary.py

Purpose: Quick audit of how much data has been collected across all sessions. Prints a table with per-session frame counts, MIDI event counts, and matched note statistics. Does not depend on any internal scripts.

Depends on: json, csv, pathlib

Used by: (standalone audit tool — no imports of project scripts)

```
python -m scripts.data_summary
```

Column	Description
Frames	Total video frames recorded in the session
MIDI evts	Total MIDI messages of all types
note_on	Key-press events with velocity > 0
MP matched	Notes where a MediaPipe fingertip landed inside the key polygon
OP matched	Notes where an OpenPose fingertip landed inside the key polygon

11. Dependency Graph

Arrows show import relationships. scripts at the top are imported by everything below them.

```
config.py
  camera_setup.py
  lux_calculator.py
  fitzpatrick_detector.py
  stats_collector.py
  midi_recorder.py
  key_calibration.py
  |   recalibrate.py
  model_manager.py
  |   live_preview.py ..... (no data written to disk)
  |   analyse.py
  |       plot_results.py
  record.py (imports all of the above)

data_summary.py (reads data/raw/ directly, imports no project scripts)
```